Further simplification of the K215 correctness proofs

Shared implication derivations, one run invariant,
and a smaller semantic interface

Research and implementation note for the ProveIt repository

21 October 2026

Abstract

The checked K215 development has already removed much of the original proof duplication. A useful next step is to reduce the number of mathematical arguments and the dependencies needed to state them. This note supplies three concrete changes against ProveIt commit `57541fe0cfe3`: a shared implication core for both the dominated and unrestricted calculi; one strengthened invariant replacing two inductions in the Lean dense-to-sparse machine bridge; and an exact extraction of the power-set predicate from a larger calibration module. The shared-core plan is attributed existing work: the new Rocq implementation connects both of its actual client families to the archived six-goal, 58-detachment derivation. The two Lean changes identify additional opportunities in the current checked tree. Focused baseline and replacement checks, typed interface audits, and recursive Rocq kernel checks are supplied. All Lean changes also compile together in a fresh 47-module closure. The combined Lean endpoint source census decreases by 53,652 bytes and two modules. These are source, interface, and dependency results; the full machine endpoints have not been rebuilt for these replacements, and no new state-count or internal-arithmetization result is claimed.

Contents

1 Recommendation and exact baseline **2**

2 One implication proof for two calculi **3**

3 A single invariant for the dense-to-sparse bridge **5**

4 Extract the power-set interface **7**

5 Verification and combined measurements **8**

6 Integration plan and reproduction **9**

7 Further research questions **10**

8 Provenance and interpretation of the contribution **11**

1 Recommendation and exact baseline

Recommended order: adopt the small interface and invariant changes first, then integrate the shared implication core in both proof assistants. Each change has a separate patch and a separate verification record. Their proof obligations are small enough to review directly, and the combined Lean check demonstrates that the delivered replacements compose.

The source snapshot examined throughout is

`57541fe0cfe3af62b4cb58747d8d1d7c16b338d5`.

The repository checkout was left unchanged. The archive contains proposals for incorporation, not a publication of changed production proof status.

1.1 Why reading the current tree matters

Recent intermediate commits describe a six-goal shared implication core and several import cuts. Subsequent merges preserve the checked checkpoint and retain some of those changes as proposals. The phase-twelve checkpoint explicitly says that the shared implication-basis and direct full-seed proposals remain separate work [1]. Therefore a commit message alone does not establish which implementation the checked endpoint currently uses.

At the pinned revision, the production generic core has 287 physical lines. Its numeric adapter has 73 lines, and the unrestricted `Lukasiewicz.lean` module has 524 lines. The latter still carries the independent 107-certificate implicational proof. The archived six-goal replacement is available under the phase-twelve upstream-merge inputs [4]. The current production Rocq pair contains 1,169 lines in `DomRho8.v` and 230 in `CD222.v`.

The current checkpoint records 179 project modules in the Lean endpoint import closure. Its accepted check freshly compiled 15 modules and retained exact dependency bindings for 169 providers in its larger 184-module primary scope. It is an incremental accepted check, not a new full build of every provider [1]. The Rocq endpoint documentation records a separate successful 237-module build and recursive kernel check with Rocq 9.1.0 [3]. Those historical results remain evidence about their own recorded sources.

1.2 The result being preserved

The machine-level objective is the equivalence

$$\mathrm{Halts}(M_{215}, \mathrm{blank}) \quad \longleftrightarrow \quad \text{there exists a finite ZFC refutation},$$

and hence, by negation,

$$\neg\,\mathrm{Halts}(M_{215}, \mathrm{blank}) \quad \longleftrightarrow \quad \mathrm{Con}(\mathrm{ZFC}).$$

The current Lean endpoint composes the literal-table relabelling, startup, runtime simulation, source-program equivalence, calculus equivalence, and ZF/ZFC calibration [2]. The proposed replacements operate inside that chain. They do not alter the 215-state table or supply a new proof that the machine actually runs forever.

There is also a distinction between an external Lean/Rocq theorem about finite ZFC proofs and an encoded ZFC or PA derivation of that theorem. The repository checkpoint expressly records this distinction. This packet preserves the former interface and does not close the latter formalization task. That qualification matters when interpreting the user's eventual requirement that the equivalence be provable *in ZFC*.

1.3 What is new here

The shared implication idea and six-goal proof plan were already present in the repository. This work contributes the concrete Rocq bridge to *both* proof families, checked together, plus a fresh check of the corresponding archived Lean proposal against the current baseline. The strengthened dense-bridge invariant and the precise `PowerSets` extraction are additional changes identified in the current tree. These contributions are implementation and proof-organization results; no new logical completeness theorem or shorter Turing machine is asserted.

2 One implication proof for two calculi

2.1 The smallest useful interface

Let A be any type, let $a \Rightarrow b$ be an arbitrary binary operation on A, and let $\preccurlyeq$ be a relation satisfying

$$\begin{aligned} u &\preccurlyeq u, \\ u \preccurlyeq x &\Longrightarrow u \preccurlyeq (x \Rightarrow y), \\ u \preccurlyeq y &\Longrightarrow u \preccurlyeq (x \Rightarrow y). \end{aligned}$$

No transitivity or antisymmetry is required. These are simply the three operations that discharge the side conditions in the checked derivation.

Write

$$R(a, b, c, u) = ((c \Rightarrow u) \Rightarrow a) \Rightarrow ((a \Rightarrow c) \Rightarrow (b \Rightarrow c)).$$

For a predicate $\mathsf{T} : A \to \mathsf{Prop}$, the *restricted interface* consists of

$$\begin{aligned} \mathsf{T}(a \Rightarrow b),\ \mathsf{T}(a) &\Longrightarrow \mathsf{T}(b), \\ u \preccurlyeq a &\Longrightarrow \mathsf{T}(R(a, b, c, u)). \end{aligned}$$

This interface is exactly the representation-independent `Language`/`Restricted` interface used in the supplied proof sources.

Theorem 2.1 (Checked shared implication basis)**.** *For every such interface, all instances of R are derivable, without the side condition $u \preccurlyeq a$. The following five additional schemata are derivable:*

$$\begin{aligned} \text{K}: &\quad a \Rightarrow (b \Rightarrow a), \\ \text{P}: &\quad ((a \Rightarrow b) \Rightarrow a) \Rightarrow a, \\ \text{B}: &\quad (a \Rightarrow b) \Rightarrow ((b \Rightarrow c) \Rightarrow (a \Rightarrow c)), \\ \text{Prefix}: &\quad (a \Rightarrow b) \Rightarrow ((x \Rightarrow a) \Rightarrow (x \Rightarrow b)), \\ \text{S}: &\quad (a \Rightarrow (b \Rightarrow c)) \Rightarrow ((a \Rightarrow b) \Rightarrow (a \Rightarrow c)). \end{aligned}$$

Proof certificate and its checking rule. The supplied plan has one restricted-axiom row and 58 condensed-detachment rows. A detachment row specifies a major and minor premise. The untrusted renderer computes their substitutions and emits a theorem whose proof applies the interface's modus-ponens field to those explicit instances. Every remaining domination premise is either a local hypothesis or follows by reflexivity and insertion into an implication operand. The Lean and Rocq kernels check the emitted terms.

Induction over the topological order of the rows proves the invariant: every emitted schema holds under exactly its displayed domination hypotheses. The axiom row follows from the restricted-row field. The induction step is modus ponens after substitution, together with the checked proofs of its

side conditions. Six designated rows have no outstanding domination hypotheses and are the schemata in the theorem. Their plan identifiers are shown in Table 1. Both renderings and their actual kernel records are included, so the finite part of this proof is supplied as executable proof terms rather than an appeal to the search program. □

Export	Plan node	Lean facade	Rocq facade
Unrestricted row	61	`rho8_all`	`rho8_all`
Composition	52	`composition`	`composition`
Prefixing schema	56	used by `lift_right`	`prefix`
K	12	`k`	`basisK`
Peirce	18	`peirce`	`basisPeirce`
S	58	`s`	`basisS`

Table 1: Goal identifiers are inherited from the archived plan and need not be consecutive. The public prefixing rule adds one modus-ponens application to an already proved implication.

The complete plan's SHA-256 is

`4141248ed90a6813c07f274351336963acde1d82f6ea463a33e5f7d64bed8005`.

The maintained source of this plan is the archived upstream proposal, not the current production three-goal plan. Its discovery and the initial generic renderer are prior work, attributed in the packet's provenance records [5, 4].

2.2 Why the abstraction is safe for numerical codes

The proof does not decode A as formulas. It does not assume that implication is injective, that different expressions have different values, or that a truth constant exists. This is useful because the machine's arithmetic codes have special aliases and need not all decode as conventional formulas. All uses of the connective are through substitution and modus ponens; arbitrary collisions in the interpretation of the connective do not invalidate those steps.

For the dominated numeric calculus, instantiate A with natural numbers, implication with `wim`, and $\preccurlyeq$ with $\leq$. The existing Cantor-pairing growth lemmas provide the two insertion laws. The old `DomImp` truth field is retained at the compatibility boundary, but the shared proof does not use it. This avoids forcing an unrelated interface migration on callers.

For the unrestricted Lean calculus, use

$$u \preccurlyeq a \quad :\Longleftrightarrow \quad \mathsf{True}.$$

Every unrestricted row then supplies a restricted row, and the relation laws are immediate. K, S, Peirce, and composition become direct instances of Theorem 2.1. The part of `Lukasiewicz.lean` after the `Tarski` namespace boundary is byte-identical to the baseline. In particular, the proof of ex falso still uses its original additional quantifier row; it is not silently inferred from positive implication alone.

2.3 The Rocq implementation and its compatibility limit

The supplied Rocq implementation has three proof files: the generic core, a numeric `DomRho8` adapter, and a `CD222` adapter. The checked adapter uses the universal relation `True`, importing the generic core directly without the numeric adapter. In it, `thm222`'s unrestricted `W_r8` supplies the restricted

row by ignoring its side-condition argument, and `W_mp` supplies modus ponens. Both proof families consequently use the same core. The four consumed `CD222` exports are

`thm_K`, `thm_P`, `thm_B`, `thm_S`.

Their exact types are checked explicitly. The unchanged `ThmDom` client is also compiled, as is a verbatim segment of the production `Prop222` development containing identity, the derivation datatype, deduction, discharge, and weakening. The client segment has a smaller import prelude; it is not represented as a build of all of `Prop222`, `G222`, `Interp222`, or `Complete222`.

The old certificate interpreter and the generated names `cd_0` through `cd_106` are removed. The selected production K215 clients do not use them. A wider consumer of those implementation names needs a migration or a compatibility file. Thus the claim is preservation of the checked public facades and their actual clients, not preservation of every accidentally exported declaration.

2.4 Source savings and what they do not measure

Selected proof sources	Before	After	Saved
Lean bytes	42,454	18,243	24,211
Lean physical lines	811	401	410
Rocq bytes	45,538	18,708	26,830
Rocq physical lines	1,399	394	1,005

Table 2: Lean counts the core plus `Lukasiewicz`; the numeric adapter is unchanged. Rocq counts both replaced files before, and the new core plus both adapters after. All new shared proof source is included.

The Lean byte reduction is 57.03%; the Rocq byte reduction is 58.92%. These counts exclude generators, test clients, dependencies, evidence, and archived inputs on both sides. They are not proof-term sizes or whole-project speed measurements. In particular, the old Rocq certificate's 735 replay instructions and the new proof's 58 detachments are different units: some explicit substitution work becomes ordinary proof-term instantiation.

The receipts record single-run timings in a shared environment; they do not establish a compilation-speed improvement. The case for the change is the smaller and shared mathematical argument. The universal-relation Rocq adapter costs nine lines and 348 bytes more than routing ordinary logic through the numeric adapter. This modest cost removes that dependency and matches the Lean instantiation. The table includes the chosen adapter and uses production `Stdlib` spelling on both sides. The checked Coq 8.18 variants are three bytes smaller on each side, with the same saving.

3 A single invariant for the dense-to-sparse bridge

3.1 The duplicated argument

`KDenseBridge.blank_iff` proves that a dense `Machine (m+1)` halts on its blank tape exactly when its relabelled sparse zipper machine halts. The baseline proof performs one time induction for the forward implication, then another to reconstruct the same correspondence for the reverse implication [6].

The first induction already has enough information to record both halting witnesses. Its terminal branch records only the sparse witness, however, so the reverse direction has to recover information that was discarded. Strengthening that branch removes the second induction without introducing a new simulation framework.

3.2 The precise invariant

Let d_t be the dense run after t steps and s_0 the sparse initial configuration. Write $R(d, z)$ for the existing tape/state correspondence, H_D for dense blank halting, and H_S for sparse halting from s_0. Maintain

$$(H_D \wedge H_S) \quad \vee \quad \exists z \, \big(\mathrm{run}_S(t, s_0) = \mathrm{some}(z) \wedge R(d_t, z)\big). \tag{1}$$

The right branch means that a corresponding sparse configuration is *present*. It does not assert that either configuration is nonterminal. In this development, the dense run may already have state `none` while the sparse run still returns `some z` for a terminal configuration. The sparse run becomes `none` only after an attempted next step. This detail prevents an off-by-one error in the argument.

Lemma 3.1 (Absorbing correspondence invariant)**.** *Under the existing row-commutation and sparse-terminal hypotheses of* `blank_iff`*, equation* (1) *holds for every* t*.*

Proof. At $t = 0$, use the existing blank-tape correspondence. Suppose the invariant holds at t. If both halting witnesses have already been obtained, retain them. Otherwise choose the corresponding configuration z. If the dense state is terminal, t witnesses H_D, and the existing `rel_halt` lemma shows that z has no sparse successor, providing H_S. If the dense state is a working state, `rel_step` supplies a sparse successor z' and the correspondence with d_{t+1}. The existing sparse `run_succ` equation gives the required present configuration at $t + 1$. □

Proposition 3.2 (Both directions from one induction)**.** *The invariant proves* $H_D \leftrightarrow H_S$ *with the original hypotheses and theorem type.*

Proof. Given a dense halting witness at time t, inspect (1). Its first branch provides H_S. Its second branch and `rel_halt` produce the sparse terminal witness at t.

Conversely, suppose the sparse run reaches y at time t and its next step is absent. Inspect the invariant at the same t. Its first branch provides H_D. In its second branch, functionality of the sparse run gives $z = y$. If d_t were a working state, `rel_step` would give a successor of y, contradicting its terminality. Therefore d_t is terminal, and t witnesses H_D. □

The replacement has the same inputs, imports, and theorem statement. Every byte before the theorem body is preserved. Only the proof body changes.

Scope	Before	After	Saved
`blank_iff` physical lines	52	37	15
`blank_iff` bytes	2,230	1,630	600
Whole module physical lines	193	178	15
Whole module bytes	8,127	7,527	600

Table 3: The replacement includes two explanatory comment lines. Its theorem body is 28.85% shorter by physical lines and 26.91% shorter by bytes.

The baseline and replacement compile with Lean 4.32.0. Both exact typed audits report precisely `[propext, Quot.sound]`. There is no additional axiom dependency. This proof pattern is useful elsewhere when two directions reconstruct the same synchronized run: preserve both terminal witnesses in the shared induction before writing a second induction.

4 Extract the power-set interface

4.1 The unnecessary dependency

At the pinned revision, `WFModelCore` imports `PSCCalibration` solely for a two-line definition:

```
def PowerSets (mem : V -> V -> Prop) : Prop :=
  forall a, exists p, forall y, mem y p <-> SetTheory.Sub mem y a
```

Here `Sub mem y a` expresses that all members of y are members of a. The two uses of the predicate occur in the hypotheses of `WFModel.domain_power` and `WFModel.domain_models_ZF`. No PSC calibration theorem is needed in that module [7, 8].

The patch creates `BusyBeaver.ZFC.PowerSets`, imports only `ZF.Zf`, and moves this exact declaration into the new file. It retains the fully qualified name

`SetTheory.BusyBeaver.ZFC.PSCCalibration.PowerSets`.

The namespace is a stable name; it need not determine which file physically contains the definition. `PSCCalibration` imports the small file for its legacy clients, while `WFModelCore` imports the small file directly.

4.2 Preservation is definitional

The new declaration has the same universe parameters, implicit carrier, membership-relation parameter, and defining expression. An explicit audit proves equality with the original defining expression by `rfl`. After removing the old declaration and changing the necessary imports, every remaining declaration and proof body in the two existing modules is byte-identical. This is a relocation of an interface, not an alternate proof of power-set existence and not an additional semantic assumption.

The typed audit also invokes `domain_models_ZF` with the original hypothesis type. Both that theorem and `psc_iff_zfAxioms` retain the recorded standard axiom profile `[propext, Classical.choice, Quot.sound]`; the definitional interface equality has no axioms.

4.3 The exact import cut

Effect on endpoint closure	Modules	Bytes
Remove `Bootstrap`	-1	$-9,174$
Remove `DefinableClosure`	-1	$-11,950$
Remove `PSCCalibration`	-1	$-8,112$
Add `PowerSets`	$+1$	$+400$
Shorter `WFModelCore` import	0	-5
Net	-2	$-28,841$

Table 4: All project roots are included in the census, including bounded-ZFC consistency. `FirstOrder.Completeness` remains required through the separate Choice chain.

The extraction alone changes the endpoint closure from 179 to 177 project modules and from 2,129,439 to 2,100,598 selected source bytes. The edited and new source files together *grow* by 320 bytes and 12 lines. The older calibration modules remain in the repository for their other clients. The improvement is a smaller dependency boundary for the endpoint, not deletion of 28,841 bytes from the repository and not a measured build-time gain.

A fresh check compiled all 29 project modules in the union of the modified `WFModelCore` closure and the compatibility `PSCCalibration` closure, followed by the typed audit. A separate reviewer independently reproduced the 179-to-177 import count and checked the exact declaration move.

5 Verification and combined measurements

5.1 What was actually executed

The archive includes the raw logs, source digests, source comparisons, compiler versions, and reproduction drivers. Table 5 distinguishes the executed scopes.

Component	Executed check	Result
Lean shared logic	16 project modules per variant; common exact-type/axiom audit; additional candidate basis audit	Pass; common audit identical
Lean dense bridge	Complete small dependency closure, original and replacement; original theorem type and axiom audit	Pass; same axiom profile
Lean power-set split	29 project modules, including legacy calibration consumer; definitional and theorem-type audit	Pass
Combined Lean	All replacements together; 47 fresh project modules and four typed audit clients	Pass
Rocq shared logic	Nine baseline and ten candidate modules; facade and real client-fragment audits; recursive `coqchk` on both	Pass; candidate 24 closed reports

Table 5: These are focused source/interface checks, not an execution of the entire modified K215 endpoint.

Lean checks use the official 4.32.0 release, commit `8c9756b28d64dab099da31a4c09229a9e6a2ef35`. The Rocq-source checks use Coq 8.18.0 with OCaml 4.14.1. In inherited production sources, the standard-library prefix `Stdlib` is normalized to `Coq` for that checker. This is disclosed in the reproduction script. A production adoption must still use the pinned Rocq 9.1 toolchain and its endpoint audit.

The generic implication exports have no axiom dependencies in either checker. Lean's numerical compatibility results retain their existing standard dependencies; the Tarski completeness result retains `propext`, `Classical.choice`, and `Quot.sound`. The common audit output is byte-identical before and after the Lean logic replacement. The Rocq candidate's 24 reports cover the generic exports, adapters, unchanged `ThmDom` result, real propositional client fragment, and explicit facade audits; every one is closed under the global context.

5.2 All changes compose

The combined check builds the union of the affected Lean closures from source in one fresh output directory, using all delivered replacements. It does not copy previously compiled project objects. Four clients then check the common logic API, the additional generic exports, the dense-bridge type, and the semantic interface. The common logic and semantic audit outputs match their earlier exact records.

The combined static endpoint census is

Metric	Baseline	Combined	Difference
Project modules	179	177	−2
Selected source bytes	2,129,439	2,075,787	−53,652

The source-byte reduction decomposes without overlap:

$$53,652 = 24,211 \text{ (logic)} + 600 \text{ (bridge)} + 28,841 \text{ (imports)}.$$

The removed modules are the three listed in the power-set table; the only added module is `PowerSets`. The Rocq source saving is separate and is not added to the Lean endpoint census.

All four proof-source patches pass `git apply -check` together against the pinned baseline. The root checkout remains clean. This establishes that the delivered edits have a concrete, consistent application point, not merely that isolated examples are plausible.

5.3 Remaining integration checks

The modified full Lean machine endpoint was not rebuilt. Nor was the modified 237-module Rocq endpoint rebuilt with Rocq 9.1. The expensive machine-specific certificates and startup proof have their existing recorded checks, but this packet does not relabel those records as evidence for new transitive dependencies. Production adoption should rebuild affected descendants and refresh source/import bindings through the repository's normal process.

No result here measures a minimum proof length, a minimum state count, a new 214-state machine, a controlled compilation speedup, or a completed internal PA/ZFC derivation. Those would require separate arguments and evidence.

6 Integration plan and reproduction

6.1 Small changes first

1. Apply the semantic-interface patch. It moves one definition and changes two import boundaries. Check the endpoint import census and legacy `PSCCalibration` consumer.

2. Apply the dense-bridge patch. Review the single strengthened invariant and its reverse-direction contradiction. Its public theorem type is unchanged.

3. Adopt the archived six-goal Lean core and the short `Lukasiewicz` adapter. Preserve the unchanged Tarski tail, keep the numeric facade, and synchronize the maintained generator with the six-goal plan.

4. Integrate the Rocq generic core and both facades. Add the new module before its consumers in the source/build order. Update `regenerate215.py` and the applicable manifests so regeneration reproduces the new source tree.

5. Rebuild the real endpoint clients with the pinned production toolchains, inspect their exact axiom reports, recursively check the Rocq endpoint, and publish new receipts bound to those exact sources.

The Rocq patch is a proof-source patch. It deliberately does not pretend that the production generator, module-order manifests, and 237-module receipts have already been updated and verified. Likewise the provided Lean patch should be incorporated together with the maintained producer, rather than allowing a subsequent regeneration to restore the older core.

6.2 Archive layout

Directory	Contents
`article/`	This LaTeX source and its compiled PDF.
`lean_logic/`	The two proposed modules, patch, typed clients, focused checker, baseline/-candidate logs.
`dense_bridge/`	Replacement module, patch, exact-type audit, reproduction driver, evidence.
`semantic_interface/`	Exact definition extraction, patch, import census, source checks, compiler evidence.
`rocq_shared/`	Shared core, both adapters, dependencies, attributed plan and renderer, patch, recursive kernel records.
`tools/`	Integrity checker, combined Lean checker, and ordered reproduction entry point.
`evidence/`	Combined check, plan identity, review notes, and integration measurements.

Compiled proof objects are omitted. The archive retains proof sources and the evidence of the actual runs. A SHA-256 inventory detects accidental changes but is not a substitute for kernel checking.

With Python 3.11 or newer, a pristine checkout at the pinned revision, Lean 4.32.0, and Coq 8.18's `coqc`/`coqchk` on `PATH`:

```
python3 tools/verify_packet.py
python3 tools/reproduce_all.py \
  --repo /absolute/path/to/ProveIt \
  --lean /absolute/path/to/lean-4.32.0/bin/lean \
  --out /tmp/k215-simplification-recheck
```

The output directory must not exist. The driver keeps the checkout and distributed evidence unchanged; a private copy is used for the semantic-interface checker. Its scope is the focused checks in this report. It does not download dependencies or launch a full machine endpoint build.

To rebuild the article:

```
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error report.tex
```

7 Further research questions

7.1 Can simulation proofs share terminal information more widely?

The dense bridge is a concrete instance of a broader review question: do the two directions of an equivalence induct over the same deterministic run while recording different terminal facts? The useful next experiment is to inspect other dense/sparse or source/runtime bridges, strengthen the shared invariant, and measure the net change in the actual consumer proofs. A new abstract framework is worthwhile only if several clients become shorter after counting that framework.

For simulations in which one source step corresponds to several target steps, retaining a terminal witness alone does not establish progress. Existing positive-step or well-founded rank conditions must remain. Allowing zero target steps indefinitely can make a nonterminating source simulate a halted target, so that generalization requires a precise progress argument.

7.2 Can the next interface cuts be found by declaration use?

The `PowerSets` example suggests a systematic search for a module imported only to obtain one definition. Import-graph analysis should be paired with a scan of actual declaration use and a compilation

of both direct and legacy consumers. A graph alone can mislead: `FirstOrder.Completeness` initially appears removable on a partial graph but remains required through the bounded-ZFC/Choice chain when every root is included.

Candidate extractions should preserve fully qualified names and definitional bodies whenever possible. This keeps compatibility checks small. The objective is to separate minimal semantic assumptions from theorems proving that a large construction satisfies them, rather than merely distributing the same dependencies among more files.

7.3 Can one plan be the maintained source for both kernels?

The same six-goal DAG now has checked renderings for Lean and Rocq. A useful maintenance task is to make the plan, its goal formulas, and its two renderers the explicit shared production input. Goal lookup should use formulas rather than incidental node numbers. Regeneration checks should reject changed exports or new assumptions, and producer/source digests should be tied to the corresponding kernel records.

This does not make the producer trusted. The kernels continue to validate ordinary proofs. It reduces the chance that a later change updates only one language or leaves a compatibility facade pointing to an obsolete derivation.

7.4 Can repeated primitive and return suffixes be shared?

The repository already contains proposals for semantic site contracts and primitive/return factoring [5, 1]. Those should be treated as existing research, not rediscovered. The next useful deliverable would replace one complete family of repeated symbolic executions with a checked positive contract, preserving exact dispatch entry, frame assumptions, exit path, and the leaf/branch coverage boundary.

A successful pilot must report both the added generic contract and the removed per-site work. It must retain the current machine-specific obligations or prove that the new contract covers them. Counting fewer generated lines without counting a new checker or a weakened hypothesis would not establish a genuine simplification.

7.5 How much of the external proof can be internalized economically?

The user's ultimate ZFC-provability objective invites a separate study of proof translation and arithmetization. The present interface reductions may help by shrinking the executable and semantic dependencies that must be represented. A concrete next question is which of the remaining correctness lemmas can be uniformly formalized in a weak arithmetic theory, and what explicit encoding is needed to turn the external equivalence into an object-theory proof.

This question should have its own theorem statement and audit. It must not be treated as automatically solved by an axiom-free external theorem or by a successful finite execution check.

8 Provenance and interpretation of the contribution

The initial inputs were obtained through the selected GitHub connection and a pinned public checkout. Existing proof plans, adapters, and producer code are attributed in the component provenance files; the earlier shared-core packet and archived upstream proposal are expressly acknowledged. The new dense-bridge invariant and exact `PowerSets` extraction were developed against the current source tree. Independent review checked the declaration move, import count, and the halting-convention detail in the invariant.

The practical contribution is a reviewable path from the current checked proofs to smaller shared arguments. It includes actual replacement sources, application patches, local proof checks, exact quantitative results, and a clear remaining production-integration boundary. The reductions do not depend on changing the mathematical meaning of the endpoint or the literal machine whose correctness is being studied.

References

[1] ProveIt, *Twelfth K215 proof-simplification checkpoint*, pinned revision `57541fe0cfe3`. Pinned checkpoint and its source/evidence links.

[2] ProveIt, `KEndpoint215.lean` and `KEndpoint.lean`, same revision. Literal 215-state endpoint; Shared K-family endpoint composition.

[3] ProveIt, *Rocq endpoints for the literal machine candidates*, same revision. Recorded production build and kernel-check scope.

[4] ProveIt, phase-twelve upstream-merge inputs, same revision. Archived six-goal plan, producer, and Lean core/adapters. Related originating commit: `3a7a61ee973a`, one 58-step derivation for both implication calculi.

[5] ProveIt, *K215: one shared implication core and semantic site contracts*, prior proposal retained at the pinned revision. Prior shared-core package and its Rocq renderer/adapter.

[6] ProveIt, `KDenseBridge.lean`, same revision. Baseline dense/sparse equivalence proof.

[7] ProveIt, `WFModelCore.lean`, same revision. Well-founded model construction and its power-set hypotheses.

[8] ProveIt, `PSCCalibration.lean`, same revision. Original power-set definition and calibration consumers.